Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 10

Disciplina: Tehnici de programare

Tema: ” Obiecte și clase ”

A efectuat : st. gr.SI – 264 Șoimaru Gabriel

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Lucrarea A

A1. Clasa Proces, un tipar de date

Un proces are patru câmpuri legate. Îl descrii ca @dataclass: numele câmpului și tipul lui.

A2. Metoda e_suspect pe Proces

Un proces care rulează din folderul utilizatorului (Temp, Roaming) e un semn clasic. Pui întrebarea ca metodă: obiectul se uită la propria cale și răspunde.

A3. Clasa Eveniment și ID-urile Windows

Evenimentele Windows au ID-uri cu sens: 4625 logon eșuat, 1102 jurnal șters, 7045 serviciu instalat. Eveniment e tot o @dataclass, cu metoda lui e_suspect.

A4. Le încarci din JSON

Scrie o funcție care citește un fișier JSON și toarnă o listă de obiecte. Proces(**d) despachetează un dicționar în câmpuri.

Lucrarea B

B1. Inventar, o clasă-container

Inventar ține lista de obiecte Proces și are un constructor alternativ, din_json, care o umple. @classmethod primește clasa (cls) în loc de un obiect.

B2. Metodele de interogare

Adaugă pe Inventar două metode: suspecte() (procesele care se declară suspecte) și pe_user() (procesele grupate pe utilizator, un dicționar de liste).

B3. Raportul JSON, din obiecte

asdict transformă un obiect înapoi în dicționar, ca să-l scrii în JSON. main() cu argparse scoate un raport de alarme: procese suspecte și evenimente de alarmă**.**

B4. Un test cu __eq__

@dataclass dă __eq__ gratis: compari obiecte după câmpuri. Scrie un test care verifică metoda e_suspect.

EFECTUAREA LUCRĂRII

Lucrarea A

A1

```python
from dataclasses import dataclass, asdict
import json
@dataclass
class Proces:
    pid: int
    nume: str
    user: str
    cale: str

p = Proces(3120, "update.exe", "ana", r"C:\Users\ana\AppData\Local\Temp\update.exe")
print(p.pid, p.nume)
print(p)
```

Figura 1.1.1 – Codul A1

```
3120 update.exe
Proces(pid=3120, nume='update.exe', user='ana', cale='C:\\Users\\ana\\AppData\\Local\\Temp\\update.exe')
```

Figura 1.1.2 – Rezultatul A1

A2

```python
CAI_SUSPECTE = ("\\Temp\\", "\\Roaming\\")

@dataclass
class Proces:
    pid: int; nume: str; user: str; cale: str
    def e_suspect(self):
        # TODO: True daca una din CAI_SUSPECTE e in self.cale
        for x in CAI_SUSPECTE:
            if x in self.cale:
                return True
        return False


p = Proces(3120, "update.exe", "ana", r"C:\Users\ana\AppData\Local\Temp\update.exe")
print(p.e_suspect()) # True (update.exe din Temp)
```

Figura 1.2.1 – Codul A2

```
True
```

Figura 1.2.2 – Rezultatul A2

A3

```python
SUSPECTE_ID = {4625, 1102, 7045}

@dataclass
class Eveniment:
    id: int; nivel: str; sursa: str; cont: str; mesaj: str
    def e_suspect(self):
        # TODO: True daca self.id in SUSPECTE_ID SAU self.nivel == "critical"
        if self.id in SUSPECTE_ID or self.nivel == "critical":
            return True
        else:
            return False

e1 = Eveniment(4625, "warning", "Security", "administrator", "Logon esuat")
print(f"Test 1 (ID 4625): {e1.e_suspect()}")

e2 = Eveniment(9999, "critical", "System", "SYSTEM", "Eroare severa")
print(f"Test 2 (Nivel critical): {e2.e_suspect()}")

e3 = Eveniment(1000, "info", "System", "user1", "Totul ok")
print(f"Test 3 (Inofensiv): {e3.e_suspect()}")
```

Figura 1.3.1 – Codul A3

```
Test 1 (ID 4625): True
Test 2 (Nivel critical): True
Test 3 (Inofensiv): False
```

Figura 1.3.2 – Rezultatul A3

A4

```python
def procese_din_json(cale):
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce [Proces(**d) for d in json.load(f)]
        return [Proces(**d) for d in json.load(f)]

procese = procese_din_json("arsenal/windows/start/procese.json")
print("procese:", len(procese))
print("suspecte:", sum(1 for p in procese if p.e_suspect()))
```

Figura 1.4.1 – Codul A4

```
procese: 7
suspecte: 2
```

Figura 1.4.2 – Rezultatul A4

Lucrarea B

B1

```python
@dataclass
class Inventar:
    procese: list
    @classmethod
    def din_json(cls, cale):
        with open(cale, encoding="utf-8") as f:
            # TODO: return cls([Proces(**d) for d in json.load(f)])
            return cls([Proces(**d) for d in json.load(f)])
    def __len__(self):
        return len(self.procese)

inv = Inventar.din_json("arsenal/windows/start/procese.json")
print("procese:", len(inv))
```

Figura 2.1.1 – Codul B1

```
procese: 7
```

Figura 2.1.2 – Rezultatul B1

B2

```python
@dataclass
class Inventar:
    procese: list
    @classmethod
    def din_json(cls, cale):
        with open(cale, encoding="utf-8") as f:
            # TODO: return cls([Proces(**d) for d in json.load(f)])
            return cls([Proces(**d) for d in json.load(f)])
    def __len__(self):
        return len(self.procese)

    #B2
    def suspecte(self):
        # TODO: lista proceselor cu p.e_suspect()
        ls = []
        for p in self.procese:
            if p.e_suspect():
                ls.append(p)
        return ls
    def pe_user(self):
        # TODO: dict user -> lista de nume (foloseste setdefault)
        users = {}
        for p in self.procese:
            users.setdefault(p.user, [])
            users[p.user].append(p.nume)
        return users

inv = Inventar.din_json("arsenal/windows/start/procese.json")
# print("procese:", len(inv))

# B2
for p in inv.suspecte():
    print(f" [{p.pid}] {p.nume} {p.cale}")
print(inv.pe_user())
```

Figura 2.2.1 – Codul B2

```
 [3120] update.exe C:\Users\ana\AppData\Local\Temp\update.exe
 [3890] svch0st.exe C:\Users\ana\AppData\Roaming\svch0st.exe
{'SYSTEM': ['System', 'svchost.exe'], 'ana': ['explorer.exe', 'chrome.exe', 'update.exe', 'powershell.exe', 'svch0st.exe']}
```

Figura 2.2.2 – Rezultatul B2

B3

```python
from dataclasses import asdict

def evenimente_din_json(cale):
    with open(cale, encoding="utf-8") as f:
        # TODO: intoarce [Proces(**d) for d in json.load(f)]
        return [Eveniment(**d) for d in json.load(f)]

def main():
    import argparse
    p = argparse.ArgumentParser()
    p.add_argument("capturi")
    p.add_argument("--json", action="store_true")
    a = p.parse_args()
    inv = Inventar.din_json(a.capturi + "/procese.json")
    ev = evenimente_din_json(a.capturi + "/evenimente.json")
    alarma = [e for e in ev if e.e_suspect()]
    # TODO: cu --json, json.dumps cu asdict pe suspecte si alarma;
    # fara, un rezumat citibil
    if a.json:
        raport = {
          "suspecte": [asdict(x) for x in inv.suspecte()],
          "alarma": [asdict(e) for e in alarma]
        }
        print(json.dumps(raport, indent=2))
    else:
        print(f"Procese: {len(inv)} | suspecte: {len(inv.suspecte())}")
        for p in inv.suspecte():
            print(f" [{p.pid}] {p.nume} {p.user} {p.cale}")
        print(f"Evenimente: {len(ev)} | alarma: {len(alarma)}")
        for e in alarma:
            print(f" {e.id} [{e.nivel}] {e.cont}: {e.mesaj}")

if __name__ == "__main__":
    main()
```

Figura 2.3.1 – Codul B3

```
Procese: 7 | suspecte: 2
 [3120] update.exe ana C:\Users\ana\AppData\Local\Temp\update.exe
 [3890] svch0st.exe ana C:\Users\ana\AppData\Roaming\svch0st.exe
Evenimente: 8 | alarma: 5
 4625 [warning] administrator: An account failed to log on
 4625 [warning] administrator: An account failed to log on
 4625 [warning] administrator: An account failed to log on
 7045 [warning] SYSTEM: A service was installed: WinUpdateSvc
 1102 [critical] administrator: The audit log was cleared
```

Figura 2.3.2 – Rezultatul B3 fără --json


```
{
  "suspecte": [
    {
      "pid": 3120,
      "nume": "update.exe",
      "user": "ana",
      "cale": "C:\\Users\\ana\\AppData\\Local\\Temp\\update.exe"
    },
    {
      "pid": 3890,
      "nume": "svch0st.exe",
      "user": "ana",
      "cale": "C:\\Users\\ana\\AppData\\Roaming\\svch0st.exe"
    }
  ],
  "alarma": [
    {
      "id": 4625,
      "nivel": "warning",
      "sursa": "Security",
      "cont": "administrator",
      "mesaj": "An account failed to log on"
    },
```


Figura 2.3.3 – Rezultatul B3 cu --json

B4

```python
from arsenal.windows.inventar import Proces

def test_proces_din_temp_e_suspect():
    p = Proces(1, "x.exe", "ana", "C:\\Users\\ana\\AppData\\Local\\Temp\\x.exe")
    assert p.e_suspect() is True

def test_proces_din_windows_nu_e_suspect():
    p = Proces(2, "svchost.exe", "SYSTEM", "C:\\Windows\\System32\\svchost.exe")
    assert p.e_suspect() is False
```

Figura 2.4.1 – Codul B4

```
..
2 passed in 0.02s
```

Figura 2.4.2 – Rezultatul B4

CONCLUZII

În această lucrare am folosit clase și obiecte pentru a automatiza detectarea activităților suspecte și să extragem eficient rapoarte de securitate din fișiere JSON, obținând un cod mult mai curat, organizat și ușor de întreținut.

BIBLIOGRAFIE

1. Python Classes - https://docs.python.org/3/tutorial/classes.html